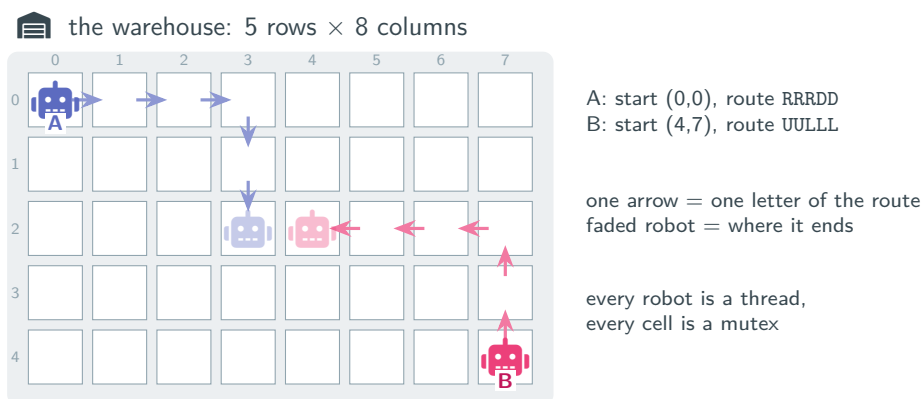


Computer Hardware ■ Lecture 8: Practice with Threads

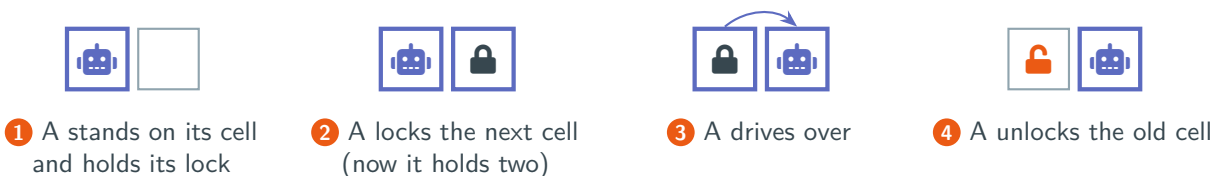
Task 4: Robots in a warehouse

A company runs an automatic warehouse: robots drive around a grid of cells. The company says that its robots can never get stuck. Prove them wrong: write routes after which the robots are stuck for ever.

The situation



Every robot is a thread, and every cell of the grid is a mutex. A robot **holds** the lock of the cell it stands on. One step takes four moments:



So a robot never stands on no cell, and two robots never stand on one cell.

The program is **ready**: you do not change the code, only the input. The routes are in the file `routes.txt`. The grid has 5 rows (0 to 4) and 8 columns (0 to 7); the cell (0,0) is in the top left corner.

```
# Routes of the robots. The grid has 5 rows (0 to 4) and 8 columns (0 to 7).
# The cell (0,0) is the top left corner.
#
# robot  row  column  route  ms per step
#   route: R = right, L = left, U = up, D = down

A  0 0  RRRDD  200
B  4 7  UULLL  300
```

In the task folder, `make run` compiles the program and runs it.

A route is a list of steps: R right, L left, U up, D down. The last number is the time of one step of this robot in milliseconds. When a robot has finished its route, it leaves the warehouse: it unlocks its cell and disappears from the grid.

After every step the program prints where the robot went and draws the grid:

```

$ ./build/robots                                # reads routes.txt
start:
  A . . . . .
  . . . . .
  . . . . .
  . . . . .
  . . . . . B

robot A: (0,0) -> (0,1)
  . A . . . . .
  . . . . .
  . . . . .
  . . . . .
  . . . . . B

...
$ ./build/robots other.txt                      # reads other.txt

```

When every robot that has not finished has waited for more than 2 s, the program prints who waits for whom and stops:

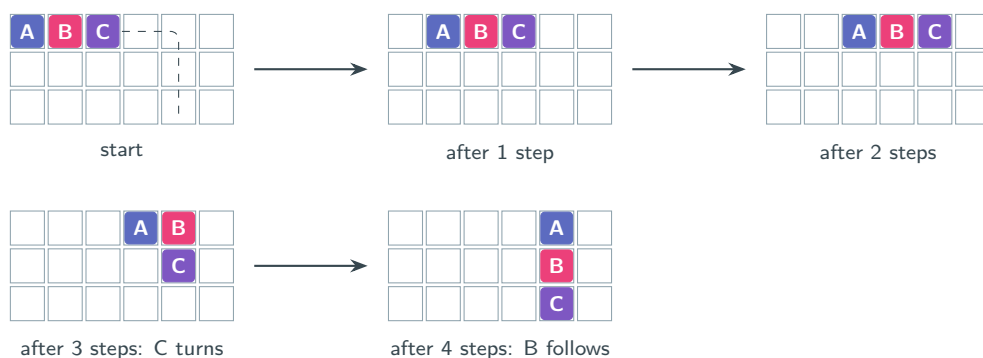
```

DEADLOCK: nobody can move.
  robot A waits for (2,4), which robot B holds
  robot B waits for (2,3), which robot A holds

```

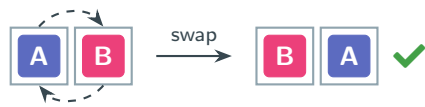
Your tasks

- 1 Write routes for 2 robots after which they are stuck for ever (a deadlock).
- 2 Does your deadlock happen every time, or only sometimes? Change the speeds of the robots and check.
- 3 Three robots drive in a line, like a snake: A right behind B, B right behind C, all on the same path. Each one may have a different speed, so a faster robot catches up with the slower one in front and waits for its cell. Can they get stuck for ever? Why? Write such routes and check.



Each robot drives into the cell that the robot in front of it has just left.

- 4 After your report, the engineers added a safety rule for the case from your report: when two robots want **each other's** cells, they **swap** places. This works only when A wants the cell of B **and** B wants the cell of A. In every other case a robot waits, as before:



A wants the cell of B,
B wants the cell of A: they swap



D wants the cell of A, but A wants
the cell of B: no swap, D waits

The boss announces that the deadlock problem is solved. Is the boss right? If not, write routes after which the robots are stuck for ever even with the safety rule. You may use as many robots as you want. To switch the rule on, add the line `safety on` at the top of your routes file.

Hint

Draw the robots on paper first: for every robot, which cell does it hold, and which cell does it want?